

SHIFA TAMEER-E-MILLAT UNIVERSITY

Department of Artificial Intelligence

PROJECT REQUIREMENTS AND ANALYSIS DOCUMENT

AI-Powered Student Event Scheduling Using Reinforcement Learning

Reinforcement Learning Semester Project

Submitted by

Muhammad Sudais Khalid

Registration Number

BSAI-23F-0050

Program

BS Artificial Intelligence, 6th Semester

Course

Reinforcement Learning

Submission Date

30 June 2026

1. Introduction

Every semester, university societies and departments run dozens of events workshops, seminars, hackathons, career fairs, guest lectures, and society meetings. Right now, almost all of that scheduling happens by hand: an organizer picks a day, checks a venue booking sheet, and hopes nothing important collides with it. This project replaces that guesswork with a system that learns. It builds a simulated university environment and trains a reinforcement learning agent to schedule events by directly optimizing for attendance, venue use, and conflict avoidance, instead of following a fixed set of rules someone wrote down once and never revisited.

The system is built around a Gymnasium-style simulation of a university semester: students with timetables and interests, events with priorities and audience estimates, venues with capacities, and an academic calendar with exam weeks and holidays. When the agent picks a day, time slot, and venue for an event, it does not just check whether the venue is free. It also looks at the weekly timetables of the students who make up the event's target audience (typically a department and semester group) and finds the slots where most of those students are actually free of lectures and labs. A venue being open means nothing if the students it is meant for are sitting in class at that time, so both checks have to pass before a slot counts as usable.

An RL agent interacts with this simulation repeatedly, and over many episodes it learns a scheduling policy that performs better than placing events at random or applying a fixed rule book.

2. Problem Statement

Current Challenges

- Two major events get scheduled at the same time, splitting the audience between them.
- Events overlap with lectures or lab sessions, so students who want to attend cannot.
- Events get planned during exam weeks, when turnout is predictably low.
- Events are planned during slots where the target students are mostly busy with lectures or labs, even when the venue itself is free.
- Venues are mismatched to event size: a 50-seat hall booked for a 300-person career fair, or a 500-seat auditorium booked for a 20-person club meeting.
- Organizers schedule based on what is convenient for them, not on what the historical attendance data suggests will work.

Problem Statement

University event scheduling is currently handled manually, based on organizer intuition and venue availability rather than on patterns in past attendance, student timetables, or academic calendar constraints. This leads to repeated scheduling conflicts, underused or overbooked venues, and inconsistent attendance. There is no system in place that learns from how previous events performed and uses that to make better scheduling decisions going forward.

3. Existing/Traditional System

Existing Process

1. An organizer or society decides they want to hold an event and picks a tentative date.
2. They manually check a shared spreadsheet or contact the venue office to see if a hall is free.
3. If the slot is free, the event is booked without cross-checking lecture timetables, exam schedules, or other events happening that week.

Limitations of Existing System

- Manual processing: every booking depends on someone remembering to check multiple sources.
- Time consuming: organizers spend significant time coordinating venues and dates by hand.
- Error-prone: clashes are usually discovered after posters are already out, not before.
- Lack of automation: nothing in the current process improves itself based on what happened last semester.
- Poor record management: attendance and feedback from past events are rarely used to inform future scheduling.

4. Motivation

Motivation Factors

- I chose this project because event scheduling is a real, recurring problem I have seen firsthand as a society president, and it maps naturally onto a sequential decision-making formulation rather than a one-shot prediction task.
- A learned scheduler can reduce clashes and wasted venue capacity without needing a human to manually re-derive rules every semester as patterns shift.
- It solves the gap between data that already exists (timetables, past attendance, venue capacity) and decisions that are currently made without referencing that data.
- Student societies, event organizers, and venue administrators all benefit from fewer clashes, better attended events, and less manual coordination.

5. Proposed Solution

The proposed system simulates a university semester and trains a reinforcement learning agent to schedule incoming event requests one at a time. Each time a new event needs a slot, the agent observes the current state of the calendar, venue availability, and the timetables of the students the event is aimed at, then chooses a day, time slot, and venue. A slot only becomes a valid action if the venue is open and it falls in a window where the common free time of the target student group is high; the agent computes this common free slot by intersecting individual student timetables for the relevant department and semester rather than treating venue availability as the only constraint. The agent is rewarded for high expected attendance and good venue fit, and penalized for conflicts, exam-week placements, overcapacity or undercapacity bookings, and slots where the target students are mostly unavailable.

Proposed Features

- A synthetic university simulator covering students, events, venues, and an academic calendar with lectures, labs, exams, and holidays.

- A common free slot calculator that intersects the weekly timetables of a department/semester group to find windows where most target students are free.
- A Gymnasium-compliant RL environment with a defined state space, action space, and reward function tailored to scheduling, where actions are restricted to slots that satisfy both venue availability and student free time.
- A trained RL agent (Q-Learning or DQN, depending on the action space size that proves workable) that schedules events better than random and rule-based baselines.
- An evaluation pipeline comparing the RL agent against a random scheduler and a rule-based scheduler on attendance, conflict rate, and venue utilization.
- A lightweight dashboard showing the learned schedule, training reward curve, and conflict statistics.

6. Project Objectives

General Objective

To design, build, and evaluate a reinforcement learning agent that learns to schedule university events more effectively than manual or rule-based methods, using a simulated university environment.

Specific Objectives

4. Formalize university event scheduling as a Markov Decision Process, including a clearly justified state space, action space, and reward function.
5. Build a synthetic simulator that generates realistic student timetables, event requests, venue constraints, and academic calendar conflicts.
6. Implement the environment in Gymnasium and train an RL agent on it, with action space reduction or masking where needed to keep training feasible.
7. Evaluate the trained agent against random and rule-based baselines using attendance, conflict count, and venue utilization as metrics.
8. Document the system design, experimental results, and limitations in a form suitable for academic defense.

7. Scope of the Project

In Scope

- A simulated university environment with synthetic student, event, venue, and calendar data.
- Computation of common free time slots across a target student group by intersecting weekly timetables, used together with venue availability to define valid scheduling actions.
- A single RL agent trained and evaluated within that simulation.
- Comparison against random and rule-based scheduling baselines.
- A basic results dashboard showing schedule output, reward curve, and conflict metrics.

Out of Scope

- Integration with a real university timetabling system or live student data.
- A full comparison across all four algorithms (Q-Learning, SARSA, DQN, PPO) at production depth; the project focuses on getting one or two algorithms working well rather than all four shallowly.
- Multi-agent or multi-university scheduling scenarios.
- Mobile application or production deployment; this remains a research prototype.

8. Stakeholders

Stakeholder	Role
Event Organizer	Submits event requests (category, expected audience, preferred timing) to the scheduling agent.
Student	Represented in the simulation through timetable, interests, and attendance probability; affected by scheduling quality.
Venue Administrator	Owns venue capacity and availability data used as environment constraints.
Course Supervisor	Evaluates the project's technical soundness and academic rigor.

9. Functional Requirements

ID	Requirement Description
FR-01	The system shall generate synthetic student, event, venue, and academic calendar data for simulation.
FR-02	The system shall represent the scheduling problem as a Gymnasium-compliant environment with <code>reset()</code> , <code>step()</code> , and a defined reward function.
FR-03	The system shall accept an incoming event request and return a scheduling decision: day, time slot, and venue, chosen only from slots where the venue is free and the target student group has high common availability.
FR-04	The system shall compute the common free time slot for a target student group by intersecting the weekly timetables of students in the relevant department and semester.
FR-05	The system shall detect and penalize scheduling conflicts, including lecture/lab overlap, exam-week placement, venue double-booking, and slots where the target students are mostly unavailable.
FR-06	The system shall train an RL agent over multiple episodes and track cumulative reward across training.
FR-07	The system shall provide a rule-based baseline scheduler and a random baseline scheduler for comparison.

ID	Requirement Description
FR-08	The system shall compute evaluation metrics: average attendance, conflict count, and venue utilization, for each scheduling method.
FR-09	The system shall display the resulting schedule, reward curve, and conflict statistics through a dashboard.

10. System Users

User Type	Responsibilities
Organizer (simulated)	Submits event requests with category, expected audience, and priority.
Administrator (project supervisor view)	Reviews schedules produced by the agent and compares them against baseline output.
RL Agent	Selects day, time slot, and venue for each incoming event based on its learned policy.

11. App Block Diagram

The system is organized in four layers. Event and calendar data flow upward from the simulator into the RL environment, decisions flow from the agent back down into the schedule, and results are surfaced through the dashboard.

University Simulator (students + timetables, events, venues, calendar)



Gymnasium RL Environment (state, action, reward)



RL Agent / Baseline Schedulers (policy, action selection)



Dashboard (schedule view, reward graph, conflict report)

12. Database Design

Storage is NoSQL, using document-style collections rather than relational tables, since the data is naturally nested (a student's timetable, an event's metadata) and the schema may still change as the simulator is refined. Tentative collections:

- students: student_id, department, semester, weekly_timetable, interests, attendance_history.
- events: event_id, category, expected_audience, duration, priority, preferred_venue, organizer.
- venues: venue_id, capacity, equipment, building, available_slots.
- calendar: type (lecture, lab, exam, holiday), date_range, affected_departments.

- `schedule_log`: `event_id`, `assigned_day`, `assigned_slot`, `assigned_venue`, `common_free_slot_score`, `reward_received`, `method` (`rl`, `random`, `rule_based`).

The common free time slot for an event's target group is not stored as its own collection. It is computed at scheduling time by pulling the `weekly_timetable` field for every student matching the event's department and semester, then intersecting those timetables to find the slots where the largest share of the group has no lecture or lab. This score feeds into the agent's action validity check alongside venue availability.

13. User Interface Mockups

Screen 1: Weekly Schedule View

Shows the agent's output for a given week as a grid of day versus time slot, with each scheduled event labeled by category and assigned venue. Conflicting placements, if any slip through, are highlighted.

Screen 2: Training Progress View

Plots cumulative reward per training episode so the learning trend is visible at a glance, alongside a simple summary of how many episodes were run and the current policy's average reward.

Screen 3: Evaluation Comparison View

A side-by-side comparison of the RL agent against the random and rule-based baselines on average attendance, conflict count, and venue utilization, using a bar chart per metric.

14. Expected Outcomes

- A working Gymnasium environment that realistically simulates university event scheduling.
- A trained RL agent that measurably outperforms random and rule-based scheduling on the chosen metrics.
- A documented comparison of scheduling methods with supporting reward and conflict data.
- A short dashboard demonstrating the agent's behavior for the project defense.

15. Future Enhancements

- Extend the comparison to DQN and PPO once the core Q-Learning or SARSA pipeline is stable, instead of attempting all algorithms in parallel.
- Replace synthetic data with real (anonymized) attendance and timetable data from the university.
- Add multi-agent scheduling to handle several societies submitting requests at once and competing for the same venues.
- Deploy the dashboard as an internal tool for the AI Innovation Society to trial on real upcoming events.

16. References

Sutton, R. S., and Barto, A. G. Reinforcement Learning: An Introduction. 2nd ed., MIT Press, 2018.

Gymnasium documentation, Farama Foundation. <https://gymnasium.farama.org>

Stable-Baselines3 documentation. <https://stable-baselines3.readthedocs.io>

Mnih, V., et al. Human-level control through deep reinforcement learning. Nature, 2015.

Schulman, J., et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.